

Final project

Databases for Developers

The final project assignment is to develop the backend part of a web application and deploy the whole solution to the cloud. You need to implement 3 solutions with following databases:

- relational database (MySQL is recommended),
- document database (MongoDB is recommended),
- graph database (Neo4j is recommended).

Recommended database cloud services: Azure MySQL server, MongoDB Atlas, Neo4j AuraDB.

Databases should be complex enough to cover the curriculum of the whole course.

The document and graph database solutions must support the same core business functionality as the relational solution. Minor differences are acceptable when caused by database-specific limitations or design trade-offs, but core CRUD operations, querying, filtering, authentication/authorization, and key business workflows must remain functionally equivalent.

Examples of the possible projects – you can come up with your own idea

- RPG – role-playing game
- Stock brokerage web app
- Logistics & Package Tracking System
- Hospital & Patient Record System
- Hotel booking & room allocation system
- Payroll & employee management system
- Restaurant ordering & kitchen workflow system
- Warehouse & inventory optimization system
- Ride-sharing & trip tracking system
- Social media content & moderation backend
- Chess platform
- Real-estate listing & rental system
- Food delivery & courier dispatch system
- Manufacturing production tracking system
- Banking system
- Movie rental / streaming system
- Car rental system
- Art collection system
- System for reviews of concerts and music releases
- Library management system

NOTE: Airline reservation system is excluded because we will use it as a class demo.

Domain model complexity

The relational database should have at least 10 main entities. Main entities refer to core business entities in the domain model (join tables, auditing, or logging do not count).

All three database solutions must be able to store the same data and query the same information – of course, the design and structure will be different.

We are working with a “monolithic” design. We deliberately avoid more complex architectures such as microservices. Instead, we want to focus on dealing with complex databases from the software developer perspective.

Substituting the frontend with Postman or Swagger UI:

The focus will be on the database and the backend server. Use Postman, Swagger UI, etc. to simulate the interaction with the API. If you happen to have a frontend, it is fine, but do not cover it in the report because it is not relevant for this course.

API documentation

Most frameworks allow you to use [Swagger](#) to create a documentation for your APIs, so you can have a swagger with 3 sets of endpoints. This will give an overview of the available database resources and actions – one set for each database. If your backend is switching between active databases it is also fine. In that case there will be only one set of endpoints.

Example with three sets of endpoints:

- .../mysql/products
- .../mongodb/products
- .../neo4j/products

Project will consist of the following:

- 1. CRUD Application (Backend)**
- 2. Relational Database**
- 3. Document Database**
- 4. Graph Database**
- 5. Migrator Application**

You are free to choose whichever tools you consider appropriate. The recommendations are MySQL as a relational database, MongoDB as a document database, and Neo4j as a graph database, because we will use them during the lectures. However, you may choose any other database of the same type if all the requirements are fulfilled.

For the CRUD application, you are free to choose any development stack – for example JavaScript or TypeScript for Node.js / Express, Java / Spring Boot, C# / ASP .NET, Python / Django / Fast API, PHP / Laravel, ...

NOTE: You must develop your own backend. You are not allowed to use firebase, supabase, or similar platforms as your API. You can use them as a cloud database service but not as an API.

Mandatory project deliverables:

- Backend CRUD application
- Relational database solution
- Document database solution
- Graph database solution
- Migrator application (one-time migration from RDBMS)
- Development environment using Docker Compose
- Cloud deployment using managed database services
- Integration tests verifying actual interaction between the application and the databases.
- AI-based data enrichment feature

Test data

At the time of delivery, the relational database must contain a realistic and meaningful amount of test data representing the selected domain. Aim for **at least 100 records per main entity** where meaningful for the domain. Test data may be generated with the help of AI tools. Only the relational database must be seeded manually, as it serves as the source system. The document and graph databases must be populated through the migrator application.

AI Integration (Data Enrichment)

The system must integrate an external AI service to generate derived data based on information stored in the database. To avoid costs, a local model can be used in the development environment. The deployed production version may have the AI feature disabled.

The AI feature must provide meaningful business value and operate on aggregated or domain-relevant data. **Simple cosmetic transformations or trivial prompt wrappers are insufficient.**

The AI feature must:

- Process and aggregate data queried from the database (e.g. reviews, descriptions, metadata)
- Persist the AI-generated result back into the database
- AI-generated data must be stored and treated as part of the domain model and must not be generated only transiently in memory.

Database user privileges

Users will be defined at database level. Define at least these users:

- A user account for the backend application which must follow the principle of least privilege and only have the minimum permissions required.
- A user with full database admin privileges
- A user with read-only privileges
- A user with restricted reading privileges that cannot access selected sensitive data

Backend Login

The system must implement authentication and authorization. Different user roles must have restricted access to endpoints and/or data according to business rules.

MySQL solution (or another relational database)

The relational database solution must implement backend CRUD functionality according to the business logic of the selected domain:

- **Querying of relational data**, including joins where appropriate
- Typical API features such as **pagination, filtering, and sorting**. Endpoints must avoid returning unlimited amounts of data.
- **Security measures** to prevent common attacks and unauthorized access to data (e.g., SQL injection prevention, input validation, parameterized queries)

MongoDB solution (or another document database)

The goal is to implement the same core functionality as the relational database while applying appropriate document modeling techniques. The design should demonstrate relevant concepts such as de-normalization, embedded documents, document references, aggregation of related data, and indexing where appropriate. Students should justify major design decisions based on query patterns and business requirements.

Neo4j solution (or another graph database)

The goal is to implement the same core functionality as the relational database while applying graph modeling principles. The design should demonstrate appropriate use of nodes, relationships, properties, graph traversals, indexing, and constraints where appropriate. Students should justify major graph modeling decisions and explain which queries benefit from graph structure.

Migrator application

A migrator application must be implemented for **one-time migration** from the relational database to both the document and graph database solutions.

The migrator will simulate a real-world scenario where an organization decides to migrate from an RDBMS to another database technology. The migrator must transfer and transform the data into MongoDB and Neo4j while preserving the required domain information and relationships. The goal is migration, not continuous synchronization between databases.

Cloud Deployment

Use managed cloud database services (e.g., Azure Database for MySQL, MongoDB Atlas, Neo4j AuraDB). Self-hosting databases in containers or virtual machines is not allowed.

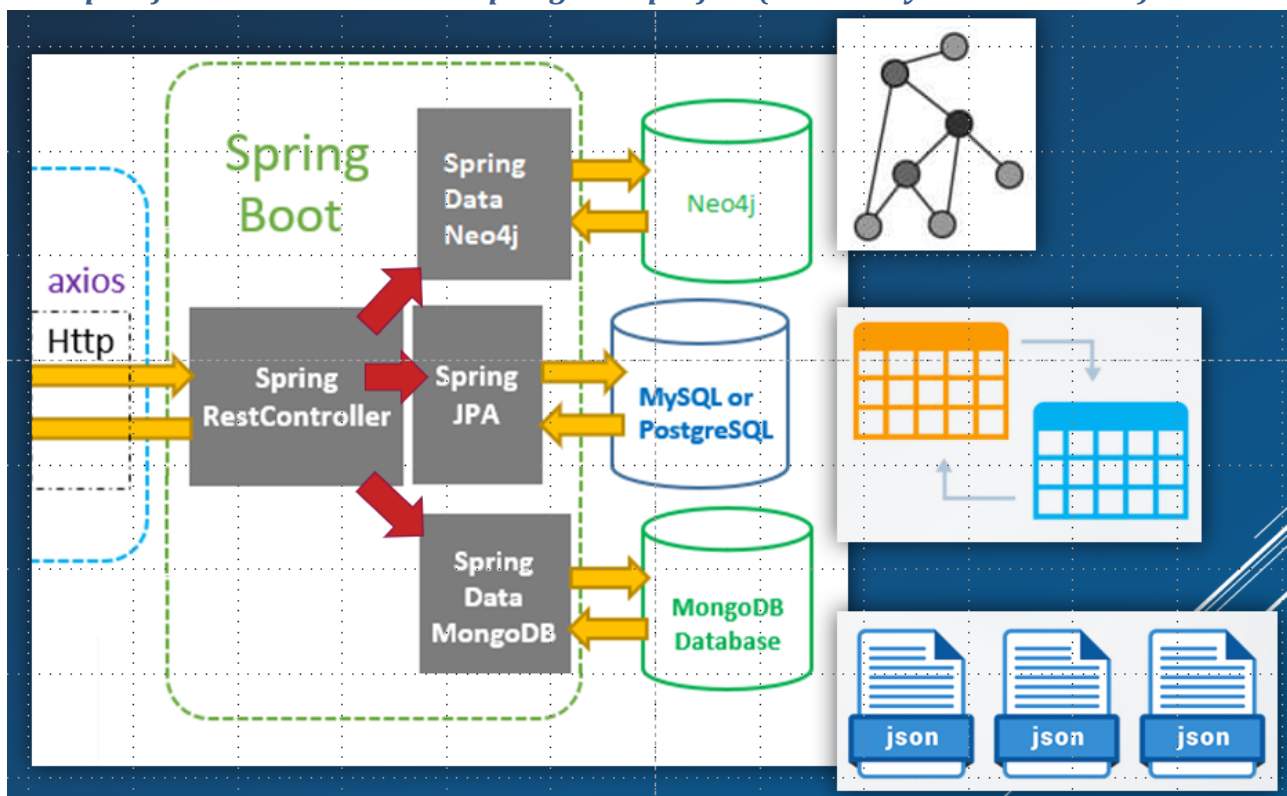
Summary

- We start with creating a system which uses an RDBMS like MySQL.
- Then we implement the same functionality with MongoDB (or similar DB).
- Then we implement the same functionality with Neo4j (or similar DB).
- Implement a migrator application for one-time migration from RDBMS and migrate.

Implementing the same functionality with different database technologies simulates a scenario when a company decides to switch from one database technology to another which we will achieve using our migrator application. That is why it is not our aim to keep the databases continuously synchronized.

It will also help to compare the strengths and weaknesses of selected database technologies.

Example of all 3 solutions in one Spring Boot project (service layer is not visible)



Apply layered architecture: controllers -> services -> models / repositories. If your backend supports generics, you can create a single set of controllers and services working with generic types and switch between the databases using a variable. Otherwise, you can make 3 sets of modules. Use DTO objects in the controllers. Do not expose the domain models to the outside.

Final Project Delivery

The final group project will be uploaded individually to WISEflow. It will consist of a report and a series of artifacts.

Final Project Artifacts – public code repository

- Relational database scripts (one or several), including:
 - Database creation, including tables, keys, indexes, constraints, and referential integrity checks.
 - Load of test data
 - Stored procedures
 - Triggers
 - Views
 - Events
 - Creation of users and privileges
- The source code of the CRUD application and the migrator - included as a link to an external public code repository (like GitHub).
- For MongoDB system:
 - dump file of the document database
 - Script for loading the test data
 - The source code of the CRUD application.
- For Neo4j system:
 - dump file of the graph database
 - Script for loading the test data
 - The source code of the CRUD application.
- The source code of the migration application (link to GitHub or another public repo).
- README file in the GitHub repo: A brief setup guide that specifies how to organize the code and import the databases in a local environment with full operational capabilities.

Final Project Report

The final report will have a big influence on the exam grade. It is the only information that is accessible to the external censor prior to the oral exam.

The report should have this structure:

Cover page, including Title, Full names of all students in the group, Group number, Date of delivery

List of figures

List of appendices

Table of contents (paginated index – the whole document should be paginated)

1. Introduction

1.1. System overview and cloud architecture: This section must include:

- Architecture diagram showing the whole system deployed in the cloud
- Diagram showing the local development deployment

1.2. Explanation of choices for databases and programming languages, and other tools.

2. Relational database

2.1. Intro to relational databases

2.2. Database design

2.2.1. Entity/Relationship Model (Conceptual -> Logical -> Physical model)

2.2.2. Normalization process

2.3. Physical data model

2.3.1. Data types

2.3.2. Primary and foreign keys

2.3.3. Indexes

2.3.4. Constraints and referential integrity

2.4. Stored objects – stored procedures / functions, views, triggers, events

2.5. Transactions. Explanation of the structure and implementation of transactions

2.6. Auditing. Explanation of the audit structure implemented with triggers

2.7. Security.

2.7.1. Explanation of users and privileges

2.7.2. SQL Injection – what is it and how it is dealt with in the project?

2.8. Description of the CRUD application for RDBMS

- Mainly the data layer – models, repositories, etc. but also briefly mention the other layers like services and controllers.
- Graphical schema of the backend modules (model, repository, service, controller, etc).
- Other relevant topics like transactions, calling the stored procedures, security, integration testing, AI integration

3. Document database

3.1. Intro to document databases

3.2. Database design – also in a graphical form showing all the collections and embeddings.

3.3. Short descriptions of features like indexes, transactions, PKs, constraints, stored objects (if stored objects are not available, how were they replaced?)

3.4. Description of the CRUD application for the document database

- same as for the RDBMS – if you have identical solution where the only difference is the data layer, you can just focus on the parts that are different.

4. Graph database

4.1. Intro to graph databases

4.2. Database design - also in a graphical form (show the database model – not a screenshot of the data)

4.3. Short descriptions of features like indexes, transactions, PKs, constraints, stored objects (if stored objects are not available, how were they replaced?)

4.4. Description of the CRUD application for the graph database

- same as for the RDBMS – if you have identical solution where the only difference is the data layer, you can just focus on the parts that are different.

5. **Migration:** Description of the migration application that migrates from a relational database into a document database and also into a graph database.

6. **Discussion:** similarities and differences between used database types, how each handled: Data modeling, Transactions, Queries, Performance, Constraints & integrity, Which database fits the domain best — and why, Trade-offs and compromises

7. **Reflection:** What was difficult about: Modeling the same domain in three databases, implementing transactions, auditing, and security, which design mistakes were made and how they were corrected, what would be done differently next time, what you learned about databases in practice.

8. **Conclusion:** short high-level summary of the project, results, main insights.

9. **References:** All major design choices, technical claims, and comparisons must be supported by references. Sources must be cited in the text and listed in the References section.

References should primarily be official documentation, textbooks, or academic or technical publications. Use standard formats like APA, IEEE, or Harvard.

10. **Appendix** - optional

Report formal requirements

You should work in groups of 4 - 5 students.

The formal requirements for the report can be found in the course catalog, in the exam section:

<https://katalog.kea.dk/course/9942152/2025-2026>